

COURSE CODE/TITLE:	CPE 010/ Data Structures and Algorithms	SECTION:	CPE21S2
DATE PERFORMED:	07/17/2026	DATE SUBMITTED:	07/17/2026
NAME:	EUNICE E. MORADILLO	INSTRUCTOR:	ENGR. JIMLORD M. QUEJADO

Hands-on Activity 2.1 Arrays, Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

SCREENSHOT

```

int main(){
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}

```

Constructor Called.
Copy Constructor is called.
Constructor Called.
Copy Assignment Operator Called.
Destructor Called.
Destructor Called.
Destructor Called.

main

0xFFFF00BA0 object Student		
student1	studentName	0xFFFF00BA0 string ?
	studentAge	0xFFFF00BA8 int ?
student2	studentName	0xFFFF00BB0 string ?
	studentAge	0xFFFF00BB8 int ?
student3	studentName	0xFFFF00BC0 string ?
	studentAge	0xFFFF00BC8 int ?

main

0xFFFF00BA0 object Student		
student1	studentName	0xFFFF00BA0 string "Roman"
	studentAge	0xFFFF00BA8 int 28
student2	studentName	0xFFFF00BB0 string ""
	studentAge	0xFFFF00BB8 int 28
student3	studentName	0xFFFF00BC0 string ""
	studentAge	0xFFFF00BC8 int 18

Student::~~Student()

OBSERVATION

Creating student1 calls the constructor once. Making student2 from student1 calls the copy constructor, not the regular constructor, since it's copying an existing object.

student3 is declared empty, so it calls the constructor again. Then student3 = student uses the copy assignment operator, not a constructor, because student3 already exists. At the end, all three objects go out of scope, so the destructor runs three times.

Table 2-1. Initial Driver Program

SCREENSHOT	<pre>int main(){ const size_t j = 5; Student studentList[j] = {}; std::string nameList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"}; int ageList [j] = {15, 16, 18, 19, 16}; return 0; }</pre> Constructor Called. Constructor Called. Constructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called.
OBSERVATION	Building the studentList array of 5 empty Student objects calls the constructor 5 times, one for each slot. Since the array is just declared and not copied from anything, no copy constructor or copy assignment is triggered here. At the end of the program, all objects are destroyed, so the destructor runs 5 times.

Table 2-2. Modified Driver Program with Student Lists

LOOP A	<pre>for(int i = 0; i < j; i++){ //loop A Student *ptr = new Student(nameList[i], ageList[i]); studentList[i] = *ptr; }</pre>
---------------	--

	<pre> Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Copy Assignment Operator Called Constructor Called. Copy Assignment Operator Called Constructor Called. Copy Assignment Operator Called Constructor Called. Copy Assignment Operator Called Constructor Called. Copy Assignment Operator Called Destructor Called. Destructor Called. Destructor Called. Destructor Called. </pre>
OBSERVATION	This loop constructs 6 new Student objects (from nameList and ageList) but also shows 4 copy assignment calls, meaning some values are being assigned into already-existing array slots rather than newly created ones. At the end, 5 destructors run as those temporary or looped objects go out of scope.
LOOP B	<pre> } for(int i = 0; i < j; i++){ //loop B studentList[i].printDetails(); } return 0; } </pre>
OBSERVATION	constructors run for new objects, and copy assignment operator runs when data is copied into existing slots (like transferring names and ages into studentList). The final output line shows the actual student data (Carly 15, Freddy 16, Sam 18, Zack 19, Cody 16), confirming the array was filled correctly. 5 destructors then clean up the objects.

OUTPUT	<pre> Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Constructor Called. Copy Assignment Operator Called Constructor Called. Copy Assignment Operator Called Constructor Called. Copy Assignment Operator Called Constructor Called. Copy Assignment Operator Called Constructor Called. Copy Assignment Operator Called Carly 15 Freddy 16 Sam 18 Zack 19 Cody 16 Destructor Called. Destructor Called. Destructor Called. Destructor Called. Destructor Called. </pre>
OBSERVATION	The output correctly prints each student's name and age (Carly 15, Freddy 16, Sam 18, Zack 19, Cody 16), matching the values originally set in nameList and ageList. This confirms that the copy assignment operator successfully transferred the data into the studentList array, since the values print correctly even though no copy constructor was used to build that array.

Table 2-3. Final Driver Program

MODIFICATIONS	<pre> //Copy Assignment Operator Student& operator=(const Student &copyStudent){ std::cout << "Copy Assignment Operator Called" << std::endl; if(this == &copyStudent){ return *this; } } </pre>
OBSERVATION	The code was missing a copy assignment operator, so I made one. Before copying any data, it checks if the object is being assigned to itself using this == &copyStudent. If that's true, it just returns the object as is instead of copying, which avoids unnecessary work and prevents potential errors that could happen from copying an object into itself.

Table 2-4. Modifications/Corrections Necessary



COMPUTER
ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```
#include <iostream>
#include <string.h>

class Grocery_Item{
protected:
    std::string itemName;
    int itemPrice, itemQuantity;
public:
    // Constructor
    Grocery_Item(std::string itemName = "None", int itemPrice = 0, int itemQuantity = 0){
        this->itemName = itemName;
        this->itemPrice = itemPrice;
        this->itemQuantity = itemQuantity;
    }
    // Virtual runs the correct version based on the object's real type,
    // even when called through a base class pointer.
    virtual int calculateSum(){
        return itemPrice * itemQuantity;
    }
    virtual ~Grocery_Item(){
        std::cout << itemName << " has been removed from memory." << std::endl;
    }
    // Copy constructor
    Grocery_Item(const Grocery_Item &sourceItem){
        std::cout << "Copy constructor called" << std::endl;
        itemName = sourceItem.itemName;
        itemPrice = sourceItem.itemPrice;
        itemQuantity = sourceItem.itemQuantity;
    }
    // Copy assignment operator
    Grocery_Item& operator=(const Grocery_Item &sourceItem){
        std::cout << "Copy Assignment operator called" << std::endl;
        if(this != &sourceItem){
            itemName = sourceItem.itemName;
            itemPrice = sourceItem.itemPrice;
            itemQuantity = sourceItem.itemQuantity;
        }
        return *this;
    }
    // Print details
    void displayInfo(){
        std::cout << "Item: " << itemName << std::endl;
        std::cout << "Price: PHP " << itemPrice << std::endl;
        std::cout << "Quantity: x" << itemQuantity << std::endl;
    }
};
```

CODE ANALYSIS:

What I did in this code is done by the Grocery_Item base class along with Fruit and Vegetable. Grocery_Item holds the shared attributes (itemName, itemPrice, itemQuantity) and shared behavior (constructor, destructor, copy constructor, copy assignment operator, and calculateSum()). Fruit and Vegetable inherit from it using public Grocery_Item, so they get all of that without rewriting it. Each still defines its own constructor, copy constructor, and copy assignment operator, but those just call the base class versions through Grocery_Item(itemName, itemPrice, itemQuantity) and Grocery_Item operator. Neither class needs its own destructor since the base class destructor already runs correctly on both, thanks to virtual. This part was actually the hardest for me to get right. Since Fruit and Vegetable don't write their own destructor, I had to make sure Grocery_Item's destructor was virtual, or deleting a Vegetable through a Grocery_Item pointer wouldn't clean up properly.

```
class Fruit : public Grocery_Item{
public:
    Fruit(std::string itemName = "None", int itemPrice = 0, int itemQuantity = 0)
        : Grocery_Item(itemName, itemPrice, itemQuantity) {}
    Fruit(const Fruit &sourceFruit) : Grocery_Item(sourceFruit){
        std::cout << "Copy constructor called" << std::endl;
    }
    Fruit& operator=(const Fruit &sourceFruit){
        std::cout << "Copy Assignment operator called" << std::endl;
        Grocery_Item::operator=(sourceFruit);
        return *this;
    }
    // No destructor needed here because the base class destructor already handles it.
};

class Vegetable : public Grocery_Item{
public:
    Vegetable(std::string itemName = "None", int itemPrice = 0, int itemQuantity = 0)
        : Grocery_Item(itemName, itemPrice, itemQuantity) {}
    Vegetable(const Vegetable &sourceVegetable) : Grocery_Item(sourceVegetable){
        std::cout << "Copy constructor called" << std::endl;
    }
    Vegetable& operator=(const Vegetable &sourceVegetable){
        std::cout << "Copy Assignment operator called" << std::endl;
        Grocery_Item::operator=(sourceVegetable);
        return *this;
    }
};
```

CODE ANALYSIS:

To this code, it defines Fruit and Vegetable, both inheriting from Grocery_Item. The constructors just pass the values up to the base class using Grocery_Item(name, price, quantity) instead of setting the attributes again. The copy constructor does the same thing, calling Grocery_Item(copyFruit) to copy everything, then printing a message. The copy assignment operator works a bit differently since it can't use that same shortcut, so it calls the Grocery_Item operator (copyFruit) inside the function body instead, then returns this. Neither class needs its own destructor since the base class one is already virtual and runs correctly on its own. Vegetable is also written the exact same way as Fruit, just with a different name. The hardest part for me was the copy assignment operator, since I couldn't reuse the initializer list trick like I did for the constructor, so I had to learn that calling Grocery_Item operator (copyFruit) directly was the correct way to reuse the base class logic there instead.

```
// Problem 3: adds up sub_total() for every item in the list
int TotalSum(Grocery_Item *GroceryList[], int size){
    int sum = 0;
    for (int i = 0; i < size; i++){
        if (GroceryList[i] != nullptr){
            sum += GroceryList[i]->calculateSum();
        }
    }
    return sum;
}
```

CODE ANALYSIS:

This function calculates the total cost of everything in the grocery list. It takes the array of pointers, GroceryList[], along with size so it knows how many slots to check. Inside the loop, it goes through each pointer one at a time and calls calculateSum() on it, adding the result to a running total called sum. Since GroceryList[i] is a pointer, -> is used to call the function on the actual object it points to. The if (GroceryList[i] != nullptr) check exists so the function doesn't try to call calculateSum() on a slot that's already been deleted, which becomes important after Problem 4 removes an item.

```
int main(){
    // array holding all items from Jenna's Grocery List
    Grocery_Item *GroceryList[4];
    GroceryList[0] = new Fruit("Apple", 10, 7);
    GroceryList[1] = new Fruit("Banana", 10, 8);
    GroceryList[2] = new Vegetable("Broccoli", 60, 12);
    GroceryList[3] = new Vegetable("Lettuce", 50, 10);

    for (int i = 0; i < 4; i++){
        GroceryList[i]->displayInfo();
        std::cout << "Subtotal: PHP " << GroceryList[i]->calculateSum() << std::endl;
        std::cout << std::endl;
    }

    int grandTotal = TotalSum(GroceryList, 4);
    std::cout << "Total Amount to Pay: PHP " << grandTotal << std::endl;

    // delete Lettuce and de-allocate its memory
    delete GroceryList[3];
    GroceryList[3] = nullptr;
    std::cout << "Lettuce has been deleted from the grocery list." << std::endl;

    return 0;
}
```

CODE ANALYSIS:

This is the main() function that will run everything. It fills the GroceryList array with Apple, Banana, Broccoli, and Lettuce using new, then loops through and prints each item's details and subtotal with displayInfo() and calculateSum(). After that, it calls TotalSum() to get the full cost and prints it. At the end, it deleted GroceryList[3] which is the lettuce.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Microsoft Copilot (BING)

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: I used AI when I tried to search general information and gather initial ideas that are related to the code that I'm working on and then AI-generated search results appeared from Bing that helped me **get a better idea** and **explained to me the concept** that helped me explain some functions in my code like fixed precision.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: AI was only used when an idea pops into my mind and try to search on Bing for reference and it did not replace my original thinking or completion of the work

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.